Workflow

The 10-stage pipeline, multi-step replanning, and the human-approval pause

1. The pipeline

A turn runs through an explicit LangGraph state machine (`app/graph/workflow.py`, nodes in `app/graph/nodes.py`). Every stage appends an event to a timeline that the chat screen renders, so what the UI shows is what actually ran.

#	Node	Responsibility	Next
1	normalize_input	Trim the query, assign request and thread identifiers, initialise the timeline	2
2	analyze_intent	Classify intent, extract entities (amounts, emails, identifiers), flag missing required fields	3
3	select_registries	Scope which namespaces the intent implies, to keep unrelated domains out of retrieval	4
4	retrieve_tools	Two-stage hybrid retrieval; returns a bounded shortlist with full parameter schemas, plus the two pinned meta-tools	5
5	validate_candidates	Candidate gate. If a mandatory field is missing, stop and ask instead of guessing	10 or 6
6	plan_and_select	Gemini function calling over the shortlist, choosing one tool and its arguments; falls back to rules when unavailable	10 or 7
7	policy_and_approval_gate	Classify risk; pause for confirmation if the action mutates external state	10 or 8
8	execute_tool	Dispatch through the execution engine with validation, idempotency header, and an audit row	9
9	evaluate_result	Decide whether the goal is met or another step is needed	6, 7 or 10
10	compose_final_answer	Synthesise the reply grounded in the execution trace; record token usage	END

2. Multi-step work

Sequences are not planned up front. After each execution, `evaluate_result` returns control to the planner, which sees the results of every completed step and either selects the next tool or answers. The loop ends when the model stops calling tools, bounded by `max_iterations`.

ERROR HANDLING
Because the planner sees prior results, a failed step is recoverable: it can read the error payload and retry with corrected arguments, or consult the RAG tool about an error code, rather than the turn simply failing.

Example - “Email me last month's sales volume”. Step 1 retrieves the transaction report. The planner then sees those figures and selects the email tool, populating the body from the returned values. Step 2 mutates external state, so it pauses for approval before sending.

Without a model configured, the rule planner uses a small set of pre-computed chains for the flows it recognises, and selects the highest-ranked candidate whose required parameters it can actually fill - so an under-specified request lists records rather than guessing an identifier.

3. The approval pause

An action is gated when any of three conditions hold: its risk tier is external-write, financial or destructive; its approval policy demands confirmation; or it is marked as mutating. The reason shown to the user names the condition that actually triggered, not a generic warning.

Suspend and resume

The gate calls LangGraph's `interrupt()`, which suspends the graph *inside the node* and persists its state through the checkpointer. Approving resumes the same run from that point, so execution continues into evaluation and any remaining steps.

```
POST /chat -> 200 status=waiting_approval, pending_approval={...}
(nothing dispatched; an approval row is pending)
POST /runs/{id}/approval -> 200 resumes the graph with the decision
action=approve execute -> evaluate -> remaining steps -> answer
action=reject cancelled, no dispatch, reason recorded
```

DURABILITY
Checkpoints are SQLite-backed, so a run awaiting a human survives a process restart. Each run gets its own checkpoint thread, so a suspended run cannot be disturbed by the next message in the same conversation.

- Only the run's owner, or an administrator, may decide an approval.
- A decision is single-use; a settled run returns HTTP 409 rather than executing again.
- If a checkpoint is genuinely unrecoverable, the run is marked expired and the user is asked to re-send, rather than the action being silently dispatched.

4. Conversation memory

Recent turns of a session are replayed into each new turn's context, which is what lets a follow-up such as “send it to john@example.com” resolve what “it” refers to. History is held in the relational store rather than in graph state, so it is independent of checkpoint lifetime and visible to the Logs and Usage screens.

5. Execution and auditing

`app/services/execution_engine.py` is the single dispatch point for every tool, whatever its kind.

- **Validation.** Required parameters are checked against the tool's contract before dispatch; a missing one is a validation error, not a malformed outbound call.
- **Idempotency.** Mutating requests carry a stable request identifier, so a retry after a timeout cannot double-charge or duplicate an invoice.
- **Routing.** Internal meta-tools, HTTP APIs and MCP servers are dispatched behind one interface, so the planner does not distinguish between them.
- **Audit.** Every attempt writes a row with the calling user, arguments, response, latency and any error - including failures, which is what makes the Logs screen trustworthy.

Error class	Handling
Missing or invalid arguments	Validation error before dispatch; returned to the planner, which can correct and retry
HTTP error from the API	Status and body captured in the audit row and surfaced to the planner
Unexpected exception	Contained, logged, and reported as a failed step; the turn continues
Model unavailable	Planner and synthesis fall back to deterministic paths; the turn still completes
Iteration budget exhausted	The loop stops and the turn answers with what it has

6. The two pinned meta-tools

Both are registered in the catalog and pinned into every shortlist regardless of retrieval score, so the agent can always consult documentation or inspect itself.

Tool	Purpose
rag_pipeline_tool	Retrieval over the indexed knowledge base: developer documentation, error codes, currency rules, and any document an administrator uploads. Retrieved passages are summarised and cited in the answer.
system_search_tool	Introspection. Scope 'capabilities' searches the tool catalog (“what tools are available for invoicing?”); scope 'execution_logs' searches run history (“what was the status of my last request?”), scoped to the calling user unless they are an administrator.